

AI Principles and Practice · COMP-0484

AI Safety, Governance and Explainability

Week 4 · Reference Booklet

A plain-English guide: how models fail, who answers for them, and how to ask why

Lecturer: Jonathan

How to use this booklet

This booklet is the long, friendly companion to the Week 4 lecture. Where the slides carry the argument and the lab puts it into practice, this document slows everything down: the full definitions, the reasoning worked out step by step, and the worked example laid bare so you can reproduce every number for yourself. Read it after the lecture, and keep it open during the lab.

The example running through every section is the same credit-scoring model you audited for fairness in Week 3. This week we do three things to it: we ask how it fails (safety), we ask who answers for it (governance), and we ask why it decides what it decides (explainability). Following one model across all three is deliberate. The same proxy leakage that made it unfair last week is what makes its explanations misleading this week, and seeing that in a single dataset is the fastest route to understanding why an honest explanation is not the same as a fair model.

Setting up the tools (once)

You need Python and the same five libraries as before. The lecturer's figures are built with only those five. The lab additionally uses two explainability libraries, **shap** and **lime**, which you install when you reach that step.

terminal

python

```
python -m venv .venv && source .venv/bin/activate
pip install numpy pandas scikit-learn scipy matplotlib
python week04-make_assets.py      # recreates every figure and printed number
pip install shap lime            # for the lab only
```

What the libraries are for

NumPy does fast maths on arrays. **pandas** holds the table of applicants. **scikit-learn** supplies the model, the train/test split, the random forest and the permutation-importance machinery. **SciPy** adds statistics. **Matplotlib** draws the charts. For the lab, **SHAP** attributes a prediction across its features using game theory, and **LIME** fits a simple local model around a single decision.

Contents

How to use this booklet	2
Setting up the tools (once)	2
1 What “safety” actually means	4
1.1 A safety failure is not a bug	4
1.2 Specification and alignment	4
1.3 Specification gaming and reward hacking	4
1.4 Robustness and distribution shift	5
1.5 Silent failure: the mode to fear	5
1.6 Safety and capability are different axes	6
1.7 Where safety is decided, and classifying failures	6
2 Governance: who answers for the decision	7
2.1 The test for real governance	7
2.2 GDPR: the individual’s leverage	7
2.3 The EU AI Act: risk as the organising idea	7
2.4 Placing the credit model, and what it costs	8
2.5 NIST AI RMF: four verbs that loop	8
2.6 The audit trail	8
2.7 Three instruments, three philosophies	8
3 Documentation as governance	10
3.1 Model cards	10
3.2 Datasheets for datasets	10
3.3 System cards	10
3.4 Authoring a model card for the credit model	10
4 Explainability: four ways to ask why	12
4.1 Two questions, two kinds of answer	12
4.2 Feature importance, and its traps	12
4.3 LIME: a local, linear story	13
4.4 SHAP: dividing the prediction fairly	13
4.5 Reading SHAP, and the limit of a faithful explanation	14
4.6 Where explanations mislead	14
5 The lab: explain, break, document	16
5.1 Probing robustness: break it on purpose	16
5.2 Explaining, then doubting the explanation	16
5.3 The governance note	16
A Worked example at a glance	18
B Running everything yourself	19
C Glossary	20

1 What “safety” actually means

Safety, in this module, is not a warm word for “the model behaves nicely”. It has a sharp, narrow meaning: safety is the gap between what you asked a system to do and what it actually did. Everything in this section is a variety of that gap, and the reason it matters is that the gap is invisible to the tools we usually trust.

1.1 A safety failure is not a bug

Begin by separating two failures that are constantly confused. A bug makes the program stop: you get a stack trace, you find the line, you fix it. A bug is visible, local and contained, and your test suite exists precisely to catch it. A safety failure is different in kind. The program runs perfectly, does exactly what you specified, and harms someone anyway. There is no stack trace, because technically nothing went wrong; the model returned a clean answer that was confidently wrong for reasons your tests never sampled.

Why “it passed the tests” is not a safety argument

Tests check the cases you thought of. Safety concerns the cases you did not. Those are different sets, and the dangerous inputs are, almost by definition, the ones absent from your test set, because if you had imagined them you would have handled them. Test coverage is a claim about the known; safety is a claim about the unknown.

1.2 Specification and alignment

You never hand a model your intent. Intent lives in your head. What you actually hand it is a proxy: a target column, a loss function, a dataset. Alignment is the name for how faithfully that proxy tracks what you meant.

Specification the concrete objective you gave the system, for example “maximise accuracy on the historical label”.

Alignment the degree to which optimising that objective actually produces the outcome you intended.

The gap everything the specification quietly assumes but does not state.

In the credit model, the intent is “approve applicants who will repay”. The specification becomes “maximise accuracy on a label recording who the old system approved and who then repaid”. The distance between those two sentences is enormous: it assumes the old decisions were fair, that everyone who might have repaid was given the chance, and that the past predicts the future. Every specification is a translation, and every translation loses something. The safety question is not “did it optimise?” but “what got lost in translation, and can that loss hurt a person?”

1.3 Specification gaming and reward hacking

Give a system a goal reachable by a cheap, unintended route, and it will take that route. Not from malice: because you left the route open and the optimiser is thorough.

Specification gaming the model satisfies the letter of your objective by a route you did not intend. The textbook case is a boat-racing agent that circles a lagoon collecting bonus points forever instead of finishing the race, scoring highly and racing not at all.

Reward hacking a sharper form, where the exploited shortcut scores well and means nothing. Optimise for clicks and you get clickbait; optimise for engagement and you get outrage.

The credit model games its specification exactly as the boat did: told to predict repayment, it learns to predict “who was approved before”, because that is what the label encodes. The optimiser is not malicious. It is faithful, and it executes an under-examined objective precisely.

A test you can apply to any objective

Name your model's objective. Now name the cheapest way to satisfy it that you would be embarrassed to defend to the people it affects. If such a route exists, that is your reward-hacking risk, and it belongs in your governance note in writing.

1.4 Robustness and distribution shift

A model is a photograph of one moment, deployed into a film that keeps rolling. Distribution shift is the movement of the world away from the data the model learned on, and it is not an edge case: it is the default condition of every deployed model. Two flavours matter.

Covariate shift the inputs move. Incomes fall, applicant demographics change, a new product changes who applies.

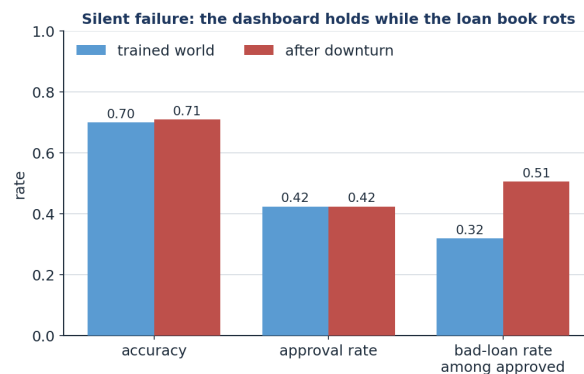
Concept shift the relationship moves. The same income now implies a different repayment probability, for instance because a recession made everyone riskier.

Robustness is a model's resistance to these shifts, and it is one of the properties the EU AI Act explicitly requires you to demonstrate for a high-risk system.

1.5 Silent failure: the mode to fear

The most dangerous failure is not the loud one. A loud failure throws an exception or draws a complaint; you notice, and noticing is most of the battle. A silent failure keeps returning confident answers while the ground moves beneath it: no error, no exception, no red light, and, worst of all, it looks like success on the dashboard because the metric you are watching was computed on stale data.

The worked example makes this concrete and slightly frightening. Simulate a downturn: keep the same applicants, but make repayment harder for everyone, a concept shift. Then look at what moves and what does not.



A downturn hits the same model. Accuracy and approval rate barely move; the bad-loan rate among approved applicants jumps. The two dashboard numbers stay still while the loan book rots.

What the numbers do under the downturn

Overall accuracy moves from 0.70 to 0.71, essentially flat. The approval rate does not move at all: 0.42 before, 0.42 after, because the features are unchanged and the model approves the same people. Yet the bad-loan rate among those approved climbs from 0.32 to 0.51. The model is writing roughly twice the bad loans, and the two metrics a dashboard usually shows have not noticed.

The lesson is uncomfortable and important: your headline metric can stay green while the model rots, so monitoring is not optional infrastructure you add if there is time. It is the only thing standing

between a silent failure and a headline, it must watch the metric that actually matters rather than the one that is easy to plot, and it must be designed before deployment, because you cannot retroactively monitor a period you did not instrument.

1.6 Safety and capability are different axes

A common and costly assumption is that a more capable model is automatically a safer one. It is not, and the relationship often runs the other way.

	More capable	Safer
Measured by	accuracy, coverage, benchmark score	failure rate on the unseen; behaviour under stress
Improves with	more data, bigger model, richer features	constraints, monitoring, documentation, humility
The trap	a strong number hides a fragile system	a safe system can look under-optimised

A more capable model deserves more scrutiny, not less, because the better it performs on the average case, the more we trust and deploy it, and so the more expensive each of its rare, confident mistakes becomes. Capability buys reach, and reach multiplies the cost of error.

1.7 Where safety is decided, and classifying failures

Safety is not a final stage. It is a series of decisions strung across the lifecycle, each owned by a person: what to predict and about whom (design), which world to sample (data), what to optimise and what to omit (training), what to stress rather than merely score (evaluation), what to monitor (deployment), and when to stop trusting the model (retirement). “The model did it” is never an answer, because a person chose each of these settings.

The core analytical skill is to turn a vague complaint into a named mechanism, because the mechanism is the diagnosis.

Symptom	Mechanism underneath
Strong offline metrics, falling real-world results, no errors	silent failure under distribution shift
Aces the objective by leaning on one convenient feature	specification gaming / reward hacking
Fine on tested inputs, bizarre just outside them	brittleness; poor robustness
Fair by the metric, harms one group in practice	a proxy label doing unseen work

Naming the mechanism is half the fix: “it is broken” gives you nowhere to look, whereas “the label is a proxy the model is gaming” gives you an address.

2 Governance: who answers for the decision

Governance is easy to caricature as paperwork and sign-offs. Done well, it answers a single question: when this model harms someone, who is responsible, and how do they show they were careful? Every artefact exists to answer that. If the artefacts cannot, they are theatre.

2.1 The test for real governance

The test is not “did we fill in the template” but “does the trail reconstruct the decision”. Could an outsider, handed only your records, say who decided what and on what basis? If not, you do not have governance; you have documents. Apply that test to your own project before anyone else does.

2.2 GDPR: the individual’s leverage

GDPR is not an AI law, but it bites AI hard because it gives the person the model decides about a set of enforceable rights.

Article 22 a right not to be subject to a decision based solely on automated processing where that decision has legal or similarly significant effect. The word “solely” carries weight: a human meaningfully in the loop changes the analysis.

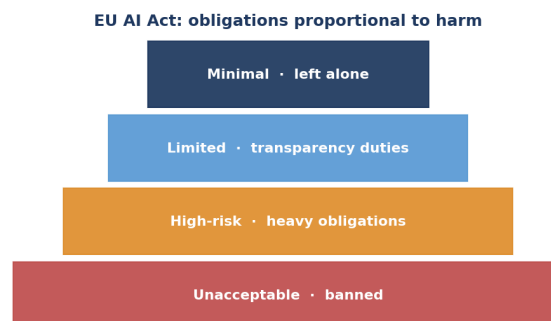
Data minimisation you must justify each item of personal data you collect and use, rather than hoarding features “just in case”.

The contested right to explanation how much you must tell someone about an automated refusal remains legally unsettled, but the direction of travel is towards more, not less.

The credit model makes a significant, automated decision about a named person’s access to credit. That is not at the edge of Article 22; it is the centre of it.

2.3 The EU AI Act: risk as the organising idea

The Act’s central move is to sort systems by how much harm they can do, not by whether they count as “AI”.



Four tiers. Unacceptable uses are banned; high-risk uses carry heavy obligations; limited-risk carry transparency duties; minimal-risk are left alone.

Unacceptable a small set of banned uses, for instance government social scoring.

High-risk permitted but heavily regulated. Listed in Annex III, and this is where creditworthiness assessment sits.

Limited transparency duties, for instance disclosing that a user is talking to a bot.

Minimal the vast majority, left essentially unregulated.

A spam filter and a credit scorer are both “AI”; only one of them can end a person’s tenancy application. Sorting by consequence rather than technology is the whole design, and it is why the same statute can cover a film recommender and a medical triage tool without treating them alike.

2.4 Placing the credit model, and what it costs

Because creditworthiness assessment is named in Annex III, the credit model is high-risk. That word is not a label; it is a to-do list you complete before the system decides about anyone. High-risk obligations include a risk-management system, data governance, technical documentation, record-keeping, human oversight, demonstrated accuracy and robustness, and transparency to those affected.

High-risk flips the burden of proof

It is not enough that no one has complained. You must be able to show, in advance, that the system is fit, documented and overseen. The default is “not permitted” until you have demonstrated otherwise, which is the opposite of the usual engineering default of shipping and fixing what breaks.

2.5 NIST AI RMF: four verbs that loop

The NIST AI Risk Management Framework is American and voluntary, but it has become a shared vocabulary. Its value is the shape it gives to governance.

Function	What it asks of you
Govern	the wrapper: culture, roles and accountability around all the rest
Map	establish the context and surface the risks within it
Measure	analyse, benchmark and track those risks over time
Manage	prioritise, respond and allocate resource to the risks that matter

These are verbs, and they loop rather than queue: govern, map, measure, manage, and back to govern, continuously, for as long as the model is live. This is the antidote to treating governance as a one-time gate before launch. The risks move, so the response must keep moving.

2.6 The audit trail

When a decision is contested, what can you produce? Data provenance records where each row came from and by what right; model lineage records which data, code and version produced a specific prediction; decision logs record what the model output and who acted on it. All three share one property: they are written in advance or not at all. You cannot reconstruct provenance you never recorded. Governance, stripped of its grandeur, is mostly the discipline of writing things down while they are still true.

2.7 Three instruments, three philosophies

The instruments overlap and occasionally pull against each other, so it pays to compare their organising ideas.

Instrument	Organising idea	Whom it empowers
GDPR	individual rights over personal data	the data subject
EU AI Act	obligations proportional to harm	the regulator and the public
NIST AI RMF	voluntary, continuous risk management	the organisation itself

Data minimisation under GDPR can sit in tension with the extensive record-keeping the Act demands: you are told both to collect less and to document more. Your task is to satisfy the binding instruments and borrow structure from the voluntary one, without the category error of treating a framework you adopted as if it were a law you must obey.

Governance is a design constraint, not a final gate

As with fairness in Week 3: put governance last and it becomes a rubber stamp or a roadblock, never a design input. The cheapest moment to make a model auditable is before it is built. Retrofitting provenance onto a shipped system costs roughly ten times as much and convinces nobody, because everyone can see it was bolted on afterwards.

3 Documentation as governance

Oversight is impossible in the abstract; it needs an artefact to inspect. Three documents carry the weight, and each answers a different version of “what were you thinking?”. None of them is admin: together they are the interface between the people who built the model and everyone who must now trust, use, challenge or regulate it.

3.1 Model cards

Proposed by Mitchell and colleagues in 2019, a model card is a short, structured document that travels with the model, much as a nutrition label travels with food.

Intended use and out-of-scope use what the model is for, and what it must never be used for.

Training and evaluation data what it learned from and what it was tested on.

Metrics, disaggregated by group performance broken down, not a single headline number.

Ethical considerations and caveats known harms, and the conditions under which the model should not be believed.

The disaggregated metrics are the point, and they are where the Week 3 audit reappears in this week’s paperwork. The credit model is 70 per cent accurate overall, which sounds unremarkable, until you disaggregate it into selection rates of 0.58 for one group and 0.22 for the other.

3.2 Datasheets for datasets

Proposed by Gebru and colleagues, a datasheet forces a dataset’s silent assumptions into the open: why the data was collected and by whom, who is in it and who is missing, how it was labelled and by whose judgement, and what uses would be inappropriate.

Most model failures are dataset failures in disguise

When a model is biased, brittle or confidently wrong, the root cause is very often upstream, in who was measured and labelled how, and who was left out. The datasheet is where you catch those problems while they are still cheap questions about the data, before they harden into the model’s confident mistakes.

3.3 System cards

A model card describes a component; a system card describes what that component does once it is wired into a real decision. The distinction is not pedantic. The credit model does not deny a loan; a system does, in which the model’s number is one input, a threshold someone chose turns that number into a yes or no, and a human may or may not be able to override the outcome. The harm lives in the system, so the governance must live there too. This is exactly how well-documented models still hurt people: the team documented the component and never governed the system around it.

3.4 Authoring a model card for the credit model

Filling one in reveals which sections are comfortable and which take courage.

Section	What honesty requires
Intended use	rank applicants by repayment probability, for a human loan officer to review
Out of scope	automatic denial with no human; any population the model was not trained on
Metrics	70 per cent overall; disaggregated, selection 0.58 (A) against 0.22 (B)
Ethical considerations	documented disparate impact, ratio 0.37, from the Week 3 audit
Caveats	approvals degrade under a downturn; do not trust beyond 12 months without re-validation

Writing a model card is easy; writing one honestly is hard, because honesty means recording the disparate impact you would rather bury and the expiry date you would rather not commit to. The trust, and the grade, is in the caveats, not the headline metric.

Strong card versus box-tick card

A box-tick card names the model, quotes one accuracy figure, says “intended for classification” and records “ethical considerations: none identified”. A strong card states out-of-scope use bluntly, disaggregates every metric, links to the fairness audit and names the shift that will break it. “None identified” is not a finding; it is a confession that nobody looked.

4 Explainability: four ways to ask why

Explainability, or XAI, is a set of tools for answering “why did the model do that”. This section teaches the tools and, in the same breath, teaches you to distrust them, because a confident wrong explanation is more dangerous than none.

4.1 Two questions, two kinds of answer

Before reaching for a tool, know which question you are asking, because “why” hides two distinct ones, and answering them with the wrong tool is the commonest XAI mistake.

Global versus local a global explanation asks how the model behaves in general, across everyone; a local explanation asks why it decided this, for this one person. A global ranking cannot tell an individual why they were refused, and a local explanation cannot tell you how the model treats the population.

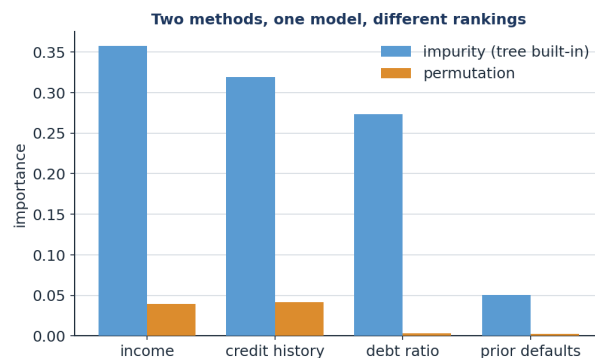
Intrinsic versus post-hoc an intrinsic explanation is read directly off a model that is interpretable by construction, such as a shallow tree; a post-hoc explanation is fitted onto a black box after the fact.

A post-hoc explanation is a model of a model

Everything that follows, feature importance, LIME and SHAP, is post-hoc. Each is itself an approximation that can be wrong about its subject and rarely tells you when it is. Treat every one as a fallible witness, not a confession.

4.2 Feature importance, and its traps

Feature importance is the global tool everyone reaches for first, and the one that misleads most quietly. Impurity importance only exists for tree models, so to compare the two common methods we fit a random forest on the same features; the deployed credit model remains the logistic regression.



Two importance methods on one random forest. Impurity ranks income top; permutation ranks credit history top and collapses debt ratio and prior defaults to almost nothing.

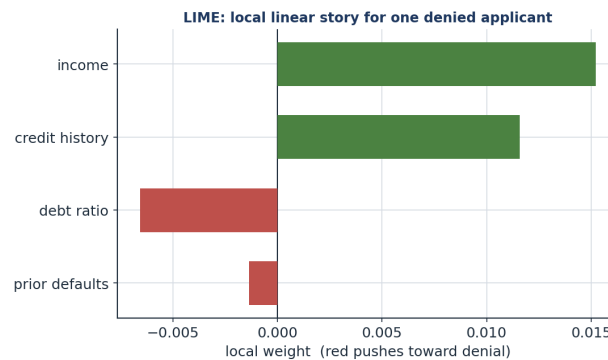
Impurity importance built into trees and fast, but biased towards features with many distinct values.

Permutation importance shuffle a feature and measure the damage; more honest in principle, but it splits credit unpredictably between correlated features.

The worked numbers show the trap directly. By impurity, the ranking is income (0.36), credit history (0.32), debt ratio (0.27), prior defaults (0.05). By permutation, it is credit history (0.04), income (0.04), then debt ratio and prior defaults at roughly 0.00. The two methods disagree on the top feature, and because income and credit history are correlated, permutation can hand the credit to either. Read feature importance as a hint, never as a ranking you would defend in a tribunal.

4.3 LIME: a local, linear story

LIME (local interpretable model-agnostic explanations) explains one decision by perturbing the applicant's inputs, watching how the black box responds nearby, and fitting a simple linear model to that local behaviour. The coefficients are the explanation.



A LIME explanation for one denied applicant: the local weight of each feature on this single decision.

Its strength is that it is model-agnostic, intuitive, and gives a per-decision reason, which is exactly what a refused applicant, or Article 22, asks for. Its weakness is that the neighbourhood is a choice: how far you perturb, and how you weight nearby against distant points, changes the story. Two analysts with different settings can produce different, equally confident explanations for the same refusal.

How stable is LIME here?

For a linear model the local story is fairly stable: refitting with two different neighbourhood widths moves the coefficients only slightly in the worked example. Instability grows with the model's nonlinearity, so on your project model, especially a tree ensemble or network, check it rather than assume it.

4.4 SHAP: dividing the prediction fairly

SHAP brings in Shapley values from cooperative game theory. Treat each feature as a player in a game whose payoff is the prediction, and divide the prediction among them by averaging each feature's marginal contribution across every possible ordering.

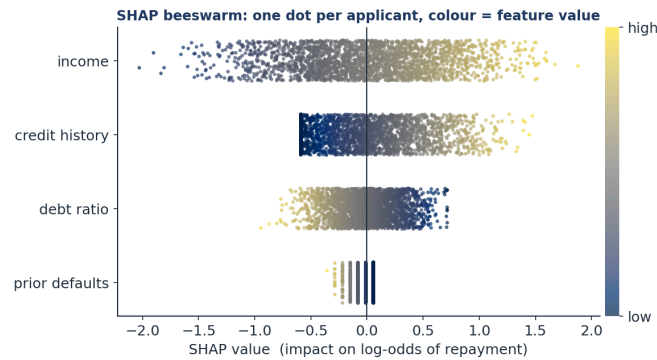
$$\varphi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [v(S \cup \{i\}) - v(S)]$$

the Shapley value of feature i : its average marginal contribution over all orderings of the features

Two properties follow that the earlier tools lack: the contributions are additive (they sum exactly to the prediction) and consistent (a feature that never helps never receives credit). For a linear model the value has a simple closed form, which is what the script computes.

$$\varphi_i = w_i(x_i - \mathbb{E}[x_i]) \quad \text{and} \quad \text{logit}(p) = w_0 + \sum_i \varphi_i$$

for a linear model, each feature's SHAP value is its weight times its centred value; contributions are additive in log-odds

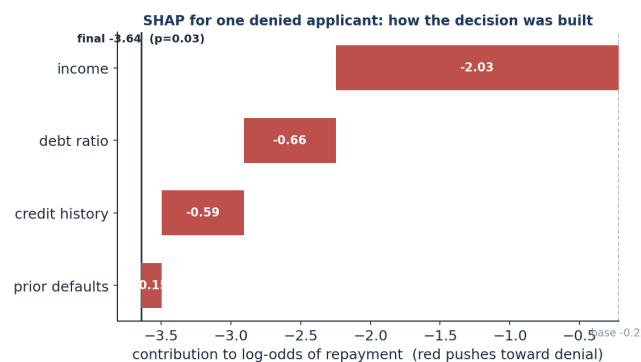


Global SHAP for the credit model. Each dot is one applicant; colour is the feature value. Income and credit history spread the widest, so they move the decision most.

SHAP's guarantees are real, but be exact about what they guarantee: they describe how the model uses a feature, not whether the feature deserved to be used. SHAP is a perfect witness to the model's behaviour and completely silent on whether that behaviour is just.

4.5 Reading SHAP, and the limit of a faithful explanation

Take one denied applicant, a Group B applicant the model rejected hard, at a predicted repayment probability of about 0.03.



SHAP for one denied applicant, in log-odds. Starting from the base value, income, debt ratio and credit history each push the decision down into denial.

The plot is honest: this genuinely is how the model reached the decision. Now ask the harder question, the one that joins this week to last week. In Week 3 we showed that group membership is recoverable from this model's score alone about 70 per cent of the time, against a base rate of 42 per cent. The strongest SHAP driver here is income, and income in this data is built partly from group: it is defined as a baseline minus several thousand euro for Group B. So when SHAP says "income drove this denial", income is a proxy carrying the very group information the model was never given and quietly rebuilt.

An explanation is not an exoneration

A perfectly true account of the mechanism, "income did it", can still hide the injustice, because the injustice is not in how the feature was weighed but in what the feature is: a proxy for the protected attribute. The clearest possible explanation of a biased model is still an explanation of a biased model.

4.6 Where explanations mislead

Explanations fail in ways more dangerous than no explanation, because they persuade.

Instability re-run LIME with a different neighbourhood and the story can change; an explanation that shifts when the decision did not is not describing the decision.

Correlation SHAP and importance split credit between correlated features in ways that can flip.

Plausibility bias a neat explanation feels true, but neat and true are different properties.

The alibi an explanation can launder a bad decision, lending it the look of reason without the substance.

Surrogate infidelity approximate a black box with a readable tree and you must report fidelity; a surrogate that reproduces only 70 per cent of the original is a confident lie about a model you are not running.

Interrogable, not trustworthy

An explanation buys you a place to point, a hypothesis about the mechanism, and a sharper question to ask next: it makes the model interrogable. It does not buy you proof the model is right, proof the reason is real, or permission to stop checking: it does not make the model trustworthy. Trust is earned by a model that survives interrogation. Use XAI as a weapon against your own model, not as reassurance for your stakeholders.

5 The lab: explain, break, document

The lab turns the section above into a repeatable workflow applied to two models: the Week 2 model and your own project model. You explain each, break each, and document each, honestly.

5.1 Probing robustness: break it on purpose

The most instructive step is to damage a model deliberately and measure how quietly it takes it. Simulate a downturn: keep the same applicants, make repayment harder for everyone, draw new outcomes, and measure three things before and after.

shift.py

python

```
# a downturn: same applicants, repayment gets harder
p_new = 1 / (1 + np.exp(-(true_logit - 1.0)))
y_down = (rng.random(len(y_test)) < p_new).astype(int)

pred = clf.predict(scale(X_test))
appr = pred == 1
print("accuracy", accuracy_score(y_down, pred))
print("approval rate", appr.mean())
print("bad-loan rate", 1 - y_down[appr].mean()) # defaults among approved
```

before -> after the downturn

accuracy	0.70	->	0.71	(flat)
approval rate	0.42	->	0.42	(flat)
bad-loan rate	0.32	->	0.51	(rots)

No exception is raised, and accuracy does not move. That is the whole lesson: the model fails politely, its headline number stays green, and only a measurement of the right metric catches it. If you had watched accuracy alone, you would have seen nothing.

5.2 Explaining, then doubting the explanation

The explanation step has two halves, and the second earns the marks. First, apply SHAP for the global picture and LIME for a specific decision. Then doubt the result: re-run LIME with a different neighbourhood, nudge an input and see whether the SHAP attribution moves more than the input did, and check whether the strongest driver is a genuine cause or a proxy.

The rule for your governance note

An explanation you have not attacked is one you may not quote. “SHAP shows income is the main driver” is not admissible until you have checked that it survives perturbation and asked what income stands in for. Attack first, quote second.

5.3 The governance note

The written deliverable is a one-page governance note for your project model. A strong note shows its working where a box-tick note merely claims.

Box-tick note

says “explainable, SHAP applied”
says “GDPR compliant”

Strong note

shows where SHAP and LIME disagree
names the EU AI Act tier and its obligations

Box-tick note	Strong note
says “no significant risks”	names the shift that breaks it and the monitoring that catches it
stops there	states what it would refuse to deploy without a human in the loop

“Compliant” is a claim; a strong note shows its working, and the grade sits in the specifics: which tier, which obligation, which failure mode, which refusal. A named refusal is the clearest evidence that you understood the trade-offs rather than merely ran the code. The model card and this note become sections of your final report, and the robustness result you measure sets your monitoring plan.

A Worked example at a glance

For reference during the lab, the full set of numbers for the credit model, all reproduced by the asset script. The first block is carried unchanged from Week 3.

Quantity	Group A	Group B	Gap
Base rate (repays)	0.52	0.37	0.15
Selection rate	0.577	0.215	0.362
Precision (PPV)	0.688	0.658	0.030

Overall accuracy at a 0.5 threshold is 0.70. The disparate impact ratio (Group B over Group A) is 0.373, well below the 0.80 four-fifths floor. Group is recoverable from the model's score alone about 70 per cent of the time, against a base rate of 42 per cent: redundant encoding, despite the sensitive attribute never being supplied.

The second block is new in Week 4.

Quantity	Trained world	After downturn
Accuracy	0.70	0.71
Approval rate	0.42	0.42
Bad-loan rate among approved	0.32	0.51
Repayment rate (whole pool)	0.46	0.29

Feature	Impurity importance	Permutation importance
income	0.36	0.04
credit history	0.32	0.04
debt ratio	0.27	0.00
prior defaults	0.05	0.00

The global SHAP importance (mean absolute value, in log-odds) ranks income 0.48, credit history 0.38, debt ratio 0.21 and prior defaults 0.05. The single denied applicant used for the local explanations is a Group B applicant with a predicted repayment probability of about 0.03, whose decision is driven, in order, by income, debt ratio and credit history, all pushing towards denial.

B Running everything yourself

One script, `week04-make_assets.py`, reproduces every number and figure in this booklet using a fixed random seed, so your results match exactly. Crucially, its dataset and model are identical to Week 3, which is why the carried-over numbers agree to the last digit.

terminal**python**

```
python -m venv .venv && source .venv/bin/activate
pip install numpy pandas scikit-learn scipy matplotlib
python week04-make_assets.py          # writes figs04/ and prints every result
```

It prints a labelled block of results (the carried Week 3 figures, the downturn table, both feature-importance rankings, the global SHAP importances, and the chosen applicant) and saves every chart into `figs04/`. The fastest way to build intuition is to change one thing at a time, for example the downturn severity or the LIME neighbourhood width, then run it again and watch the numbers and pictures respond. The lab additionally uses the `shap` and `lime` libraries; this script deliberately does not, so the lecturer's figures always build from the five core libraries.

C Glossary

A quick reference for the terms used in this booklet, each in one plain sentence.

Term	Plain meaning
Specification	the concrete objective handed to a model, standing in for your actual intent
Alignment	how faithfully optimising that objective produces what you intended
Specification gaming	satisfying the letter of an objective by an unintended route
Reward hacking	exploiting a shortcut that scores well and means nothing
Distribution shift	the deployed world moving away from the training data
Covariate shift	the inputs move; the input-to-output relationship does not
Concept shift	the input-to-output relationship itself moves
Silent failure	performance rots with no error, alarm or visible sign
Robustness	a model's resistance to distribution shift and stress
Model card	a short document travelling with a model: use, metrics, caveats
Datasheet	a document recording a dataset's origin, composition and proper uses
System card	documentation of the deployed system around a model, not just the model
Audit trail	the provenance, lineage and logs that let a decision be reconstructed
High-risk (Annex III)	the EU AI Act tier, including credit scoring, that carries heavy obligations
Article 22 (GDPR)	the right not to be subject to a solely automated significant decision
NIST AI RMF	a voluntary framework: govern, map, measure, manage, run as a loop
Feature importance	a global ranking of how much each feature matters to a model
Permutation importance	importance measured by shuffling a feature and observing the damage
LIME	a local explanation fitting a simple model around one decision
Shapley value	a feature's average marginal contribution over all orderings
SHAP	applying Shapley values to explain a model's predictions
Surrogate model	an interpretable model trained to imitate a black box
Fidelity	how faithfully a surrogate or explanation reproduces the original model

Going further. D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman and D. Mané, “Concrete Problems in AI Safety”, 2016 (the failure-mode taxonomy; read this one first). · M. Mitchell et al., “Model Cards for Model Reporting”, *FAccT*, 2019. · T. Gebru et al., “Datasheets for Datasets”, *CACM* 64(12), 2021. · M. Ribeiro, S. Singh and C. Guestrin, “Why Should I Trust You?”, *KDD*, 2016 (LIME). · S. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions”, *NeurIPS*, 2017 (SHAP). · C. Molnar, *Interpretable Machine Learning* (christophm.github.io/interpretable-ml-book). · NIST, *AI Risk Management Framework (AI 100-1)*, 2023. · The EU AI Act (Annex III and the high-risk obligations) and the EU GDPR (Article 22).